Space Empire

Gestion des queues

Construction · Recherche · Bâtiments

Mars 2026

1. Contexte et périmètre

Le jeu dispose de trois types de queues indépendantes par planète : construction de vaisseaux (shipyard), recherche technologique (research) et amélioration de bâtiments (building). Chaque queue fonctionne selon les mêmes règles et le même modèle de données.

Les décisions de conception prises dans ce document visent à :

- Simplifier le modèle de données tout en couvrant les cas métier réels
- Gérer correctement les suppressions en cascade (dépendances entre items)
- Supporter les boosts de vitesse cumulables sans complexité excessive
- Intégrer proprement RabbitMQ pour les événements différés

2. Modèle de données

2.1 Entité QueueItem

Une seule entité couvre les trois types de queues. Le champ queue_type différencie shipyard, research et building.

Champ	Type	Rôle
`id`	`int`	Clé primaire — sert aussi d'ordre de queue (ORDER BY id ASC)
`planet`	`Planet`	Planète propriétaire de la queue

Champ	Type	Rôle
`queue_type`	`string`	'shipyard' | 'research' | 'building'
`item_slug`	`string`	Référence vers le catalogue YAML
`item_level`	`int|null`	Niveau cible (bâtiments et technologies), null pour les vaisseaux
`quantity`	`int`	Quantité (vaisseaux), 1 pour les autres types
`duration_seconds`	`int`	Durée calculée à l'ajout. Modifiée directement par les boosts.
`started_at`	`datetime|null`	Renseigné uniquement pour l'item en tête de queue. null si en attente.
`queued_at`	`datetime`	Horodatage d'ajout dans la queue (informatif)
`dispatch_token`	`string`	UUID régénéré à chaque redispatch RabbitMQ (voir section 4)
`depends_on`	`QueueItem|null`	FK nullable vers un autre QueueItem. Suppression en cascade.

2.2 Champs sur Planet

Chaque type de queue a son propre T0 (moment de démarrage du premier item). Trois champs distincts sur Planet :

Champ	Rôle
`shipyard_queue_started_at`	T0 de la queue vaisseaux. null si queue vide.
`research_queue_started_at`	T0 de la queue recherche. null si queue vide.
`building_queue_started_at`	T0 de la queue bâtiments. null si queue vide.

2.3 Calcul du finish_at

> *finish_at n'est jamais persisté en base de données. Il est toujours calculé à la volée à partir de queue_started_at et des duration_seconds des items précédents.*

Formule :

```
finish_at(item N) = queue_started_at + sum(duration_seconds des items 1..N)
```

Exemple avec trois items en queue :

```
Queue : [A(60s), B(90s), C(120s)]
finish_at(A) = T0 + 60
```

```
finish_at(B) = T0 + 150
finish_at(C) = T0 + 270
```

2.4 Convention d'ordre

L'ordre de la queue est déterminé par l'id ASC. Il n'y a pas de champ position séparé.

> *Convention à documenter dans le code : toujours utiliser ORDER BY id ASC pour récupérer la queue. Ne jamais assumer un ordre sans ce tri explicite.*

```
$queueItems = $this->queueItemRepository->findBy(
    ['planet' => $planet, 'queue_type' => $queueType],
    ['id' => 'ASC']
);
```

3. Prérequis et ajout en queue

Les prérequis sont vérifiés uniquement au moment de l'ajout. Il n'y a pas de vérification différée ni d'anticipation sur des items en cours de traitement.

> *Si les prérequis ne sont pas satisfaits au moment du lancement (technologie pas encore recherchée, bâtiment pas encore construit), l'ajout est refusé. On ne peut pas anticiper une construction qui dépend d'une recherche en cours.*

Conséquence UX à documenter : le joueur qui veut enchaîner une recherche et une construction dépendante de cette recherche doit attendre que la recherche soit terminée avant de lancer la construction. C'est un choix de game design assumé, pas une limitation technique.

4. Dépendances entre items

4.1 Cas d'usage

Un item peut dépendre d'un autre item de la même queue. Exemple typique :

- Le joueur améliore son Bâtiment au niveau 3 (item en queue, position 1)
- Il lance immédiatement l'amélioration au niveau 4 (item en queue, position 2)
- Le niveau 4 est valide car le niveau 3 est en cours et sera satisfait avant

Avec depends_on, si le niveau 3 est supprimé, le niveau 4 est automatiquement supprimé en cascade.

4.2 Suppression en cascade

La suppression est récursive pour couvrir les chaînes longues (lvl3 → lvl4 → lvl5) :

```
public function removeItem(QueueItem $item): void
{
    // Trouver les items qui dépendent de celui-ci
    $dependents = $this->queueItemRepository
        ->findBy(['depends_on' => $item]);

    foreach ($dependents as $dependent) {
        $this->removeItem($dependent); // récursif
    }

    $this->doRemoveItem($item);
}
```

4.3 Exemple concret

Queue : [Bât. lvl3 (id=10), Bât. lvl4 (id=11, depends_on=10), Autre (id=12)]

Avant suppression lvl3	Après suppression lvl3
id=10 : Bât. lvl3 (60s)	❌ supprimé
id=11 : Bât. lvl4 (90s, depends_on=10)	❌ supprimé en cascade
id=12 : Autre (45s)	☑ inchangé — devient tête de queue

5. Boosts de vitesse

5.1 Principe

Un boost modifie directement duration_seconds. Il n'y a pas de champ séparé. Les boosts sont cumulables : chaque boost s'applique sur la valeur courante de duration_seconds.

Étape	duration_seconds
Valeur initiale	`3600s`

Étape	duration_seconds
Boost x2 appliqué	`1800s`
Boost x2 appliqué à nouveau	`900s`

5.2 Le boost s'applique sur le temps restant

Si le boost arrive en cours d'exécution, il s'applique sur le temps restant uniquement, pas sur la durée totale.

> *Exemple : item démarré à T=0, duration_seconds=3600. Boost x2 appliqué à T=1200. elapsed = 1200s | remaining = 2400s | new remaining = 1200s new duration_seconds = 1200 + 1200 = 2400s*

```
$elapsed = $now->getTimestamp() - $item->getStartedAt()->getTimestamp();
$remaining = $item->getDurationSeconds() - $elapsed;
$newRemaining = (int) ceil($remaining / $divisor);
$item->setDurationSeconds($elapsed + $newRemaining);
```

5.3 Impact sur les items suivants

Les items en attente (pas encore en tête de queue) ne sont pas affectés directement. Leurs duration_seconds restent intacts. Leurs finish_at se recalculent automatiquement puisque l'item actif se termine plus tôt.

> *Avant boost : [Item1 reste 2400s | Item2 900s | Item3 600s] finish_at : T+2400 | T+3300 | T+3900 Après boost x2 sur Item1 : [Item1 reste 1200s | Item2 900s | Item3 600s] finish_at : T+1200 | T+2100 | T+2700 Item2 et Item3 : duration_seconds inchangés, finish_at recalculés automatiquement.*

6. RabbitMQ — Option B retenue

6.1 Pourquoi on ne modifie pas les messages existants

Modifier un message déjà dispatché avec DelayStamp est impossible nativement dans RabbitMQ. Les alternatives (RabbitMQ Management API, plugin spécifique) sont complexes et couplent le code à l'infrastructure.

L'option retenue : dispatcher un nouveau message avec le bon délai et ignorer l'ancien à son arrivée via un dispatch_token.

6.2 Le dispatch_token

Le dispatch_token est un UUID stocké sur QueueItem et inclus dans le message RabbitMQ. À chaque redispatch (boost, annulation), le token est régénéré. Le Handler compare le token du message avec celui en base et ignore le message si les deux ne correspondent pas.

Cas	Sans dispatch_token	Avec dispatch_token
Item supprimé	item === null suffit	Double protection
Boost appliqué	❌ ancien message traite l'item	☑ message obsolète ignoré
Double-dispatch accidentel	❌ traitement en double	☑ doublon ignoré

6.3 Structure du message

```
class QueueItemFinished
{
    public function __construct(
        public readonly int    $queueItemId,
        public readonly string $dispatchToken,
    ) {}
}
```

6.4 Handler

```
public function __invoke(QueueItemFinished $message): void
{
    $item = $this->queueItemRepository->find($message->queueItemId);

    // Cas 1 : item supprimé entre-temps
    if (!$item) {
        return;
    }

    // Cas 2 : token obsolète (boost ou double-dispatch)
    if ($item->getDispatchToken() !== $message->dispatchToken) {
        return;
    }

    // Cas 3 : traitement normal
    $this->queueService->completeItem($item);
}
```

6.5 Flux lors d'un boost

```
// 1. Calculer le nouveau temps restant
$newRemaining = (int) ceil($remaining / $divisor);
$item->setDurationSeconds($elapsed + $newRemaining);

// 2. Régénérer le token — l'ancien message deviendra obsolète
$newToken = Uuid::uuid4()->toString();
$item->setDispatchToken($newToken);
$this->queueItemRepository->save($item, true);

// 3. Dispatcher un nouveau message avec le délai correct
$this->messageBus->dispatch(
    new QueueItemFinished($item->getId(), $newToken),
    [new DelayStamp($newRemaining * 1000)]
);
// L'ancien message arrivera mais sera ignoré (token différent)
```

7. Récapitulatif des décisions

Sujet	Décision retenue
finish_at	Jamais persisté — calculé à la volée
Ordre de queue	ORDER BY id ASC — pas de champ position
Multi-dépendances	Non retenu — FK simple depends_on nullable
Prérequis	Vérifiés à l'ajout uniquement, jamais en différé
Boosts	Modifient duration_seconds directement, cumulables
Boost sur items en attente	Non — uniquement sur l'item actif
RabbitMQ — modification message	Option B — nouveau message + dispatch_token
Queues par planète	3 queues indépendantes : shipyard, research, building

Space Empire — Gestion des queues — Mars 2026